



TAREA

#2 (PATRONES)

TEMA:

SISTEMA SPORTSPREDICTOR

GRUPO:

5

INTEGRANTES:

NAVARRETE FIGUEROA JOSÉ ANTONIO

ZOUEIN VÉLEZ NEJEH YOUSSEF

INGA ONTANEDA ANGIE LILIANA

BORBOR CRESPIAN KEVIN ANDRES

ASIGNATURA:

DISEÑO DE SOFTWARE

PARALELO:

P1

PROFESOR:

Mgtr. JURADO MOSQUERA DAVID ALONSO

Índices

1. Patrones de diseño seleccionados.....	3
1.1. Patrones de comportamiento.....	3
1.1.1. Chain of Responsibility.....	3
1.1.2. Observer.....	3
1.2. Patrón estructural.....	3
1.2.1. Adapter.....	3
1.3. Patrón creacional.....	4
1.3.1. Factory Method.....	4
2. Diagrama de clases.....	4

1. Patrones de diseño seleccionados

1.1. Patrones de comportamiento

1.1.1. Chain of Responsibility

Se selecciona Chain of Responsibility porque los reportes de discrepancia pueden pasar por diferentes niveles de atención: primero el equipo de soporte, y si el caso requiere una revisión más profunda, se escala a control de calidad.

Esto se modela con manejadores conectados en cadena, donde cada uno decide si resuelve el incidente o lo pasa al siguiente nivel, sin que el sistema necesite conocer de antemano cuántos niveles existen. Así, agregar un nuevo nivel de escalamiento en el futuro solo implica sumarlo a la cadena, sin modificar los niveles ya existentes.

1.1.2. Observer

Se selecciona Observer porque se especifica que se debe notificar a los usuarios sobre los estados de los pronósticos, los resultados de los eventos y los cambios en las puntuaciones de los usuarios; siendo este enviado mediante correo electrónico o aplicaciones de mensajería.

Aplicar este patrón permite separar la lógica de negocio de los mecanismos de envío de alertas, de modo que cada canal de notificación se encarga solo de su propia tarea. Esto reduce el acoplamiento entre clases, ya que el negocio solo depende de la interfaz `CanalNotificacion` y no de sus implementaciones concretas, y facilita agregar nuevos canales de notificación en el futuro sin modificar el código existente.

1.2. Patrón estructural

1.2.1. Adapter

Se selecciona Adapter debido a que el sistema necesita mostrar estadísticas de jugadores, tendencias de equipos y datos históricos, información que típicamente proviene de fuentes externas con formatos distintos entre sí.

Para esto, se define una interfaz común que el sistema usa internamente para consultar estadísticas, y un adaptador por cada proveedor externo que traduce su formato particular a dicha interfaz. Esto reduce el acoplamiento con servicios externos y facilita incorporar nuevos proveedores deportivos agregando simplemente un nuevo adaptador, sin modificar el resto del sistema.

1.3. Patrón creacional

1.3.1. Factory Method

Se selecciona Factory Method porque el sistema necesita crear distintos tipos de pronósticos según el deporte: todos comparten acciones como consultar su estado o calcular puntos, pero cada uno maneja información distinta (marcador exacto en fútbol, puntos totales en baloncesto, sets ganados en tenis).

Si una sola clase decidiera con if o switch qué pronóstico crear, tendría que conocer todos los deportes y modificarse cada vez que se agregue uno nuevo. Esto se resuelve dando a cada deporte su propia fábrica encargada de crear el pronóstico correspondiente, de modo que agregar un nuevo deporte solo implica crear una nueva fábrica, sin tocar el código existente.

2. Diagrama de clases

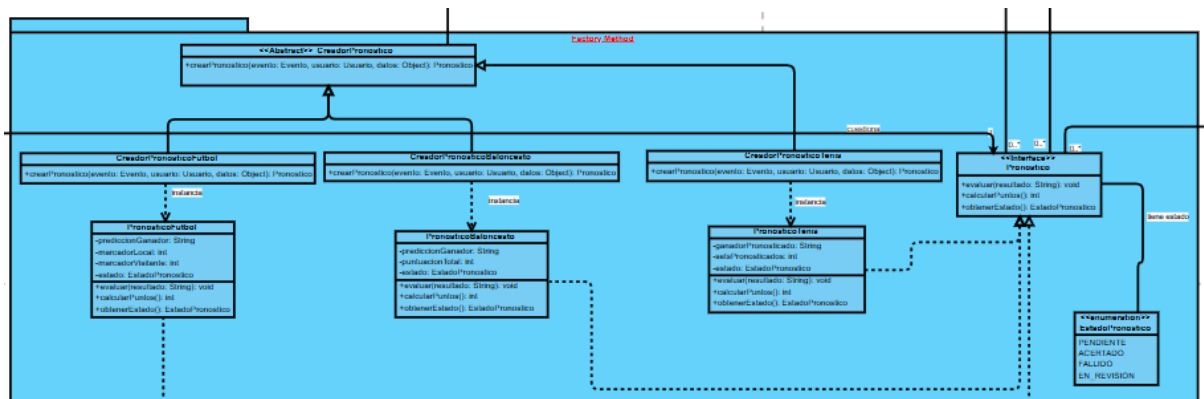


Figura 1: Factory Method

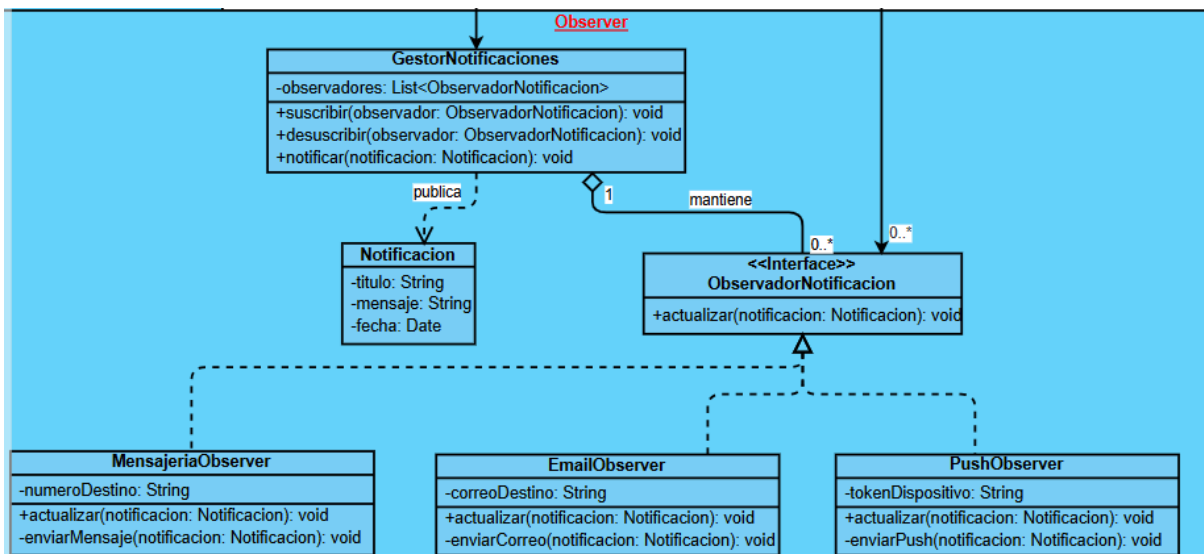


Figura 2: Observer

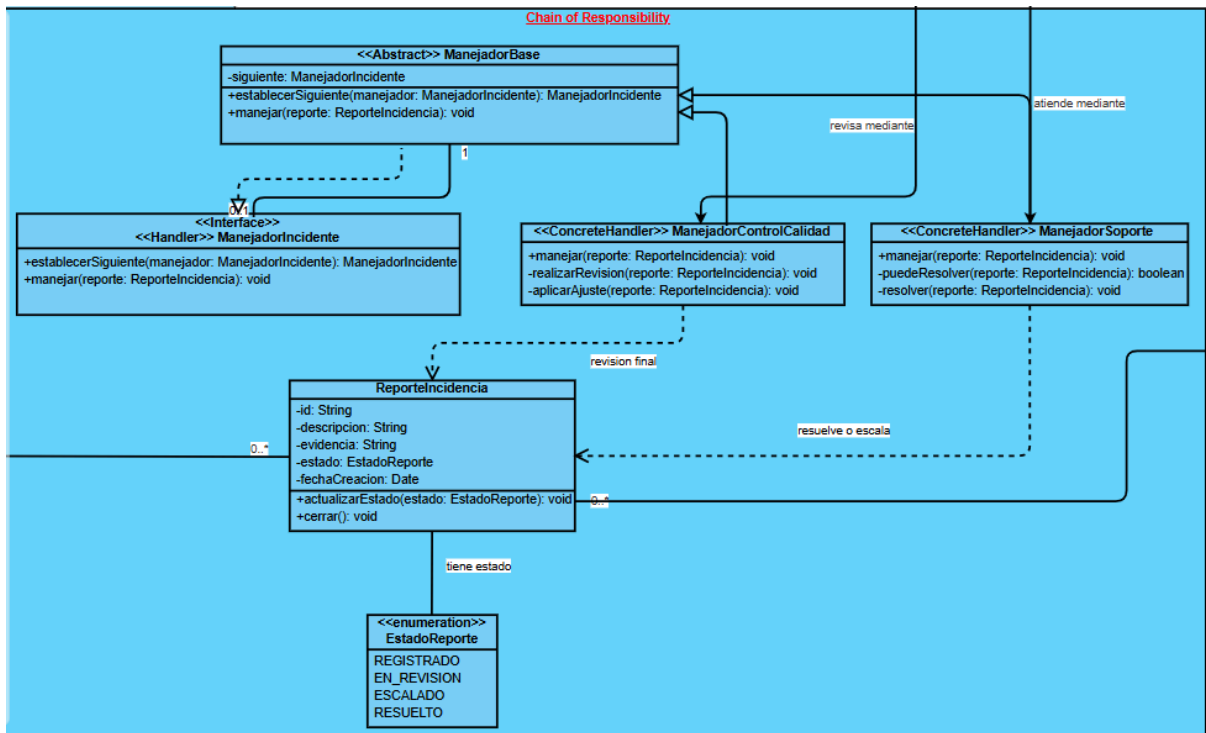


Figura 3: Chain of Responsibility

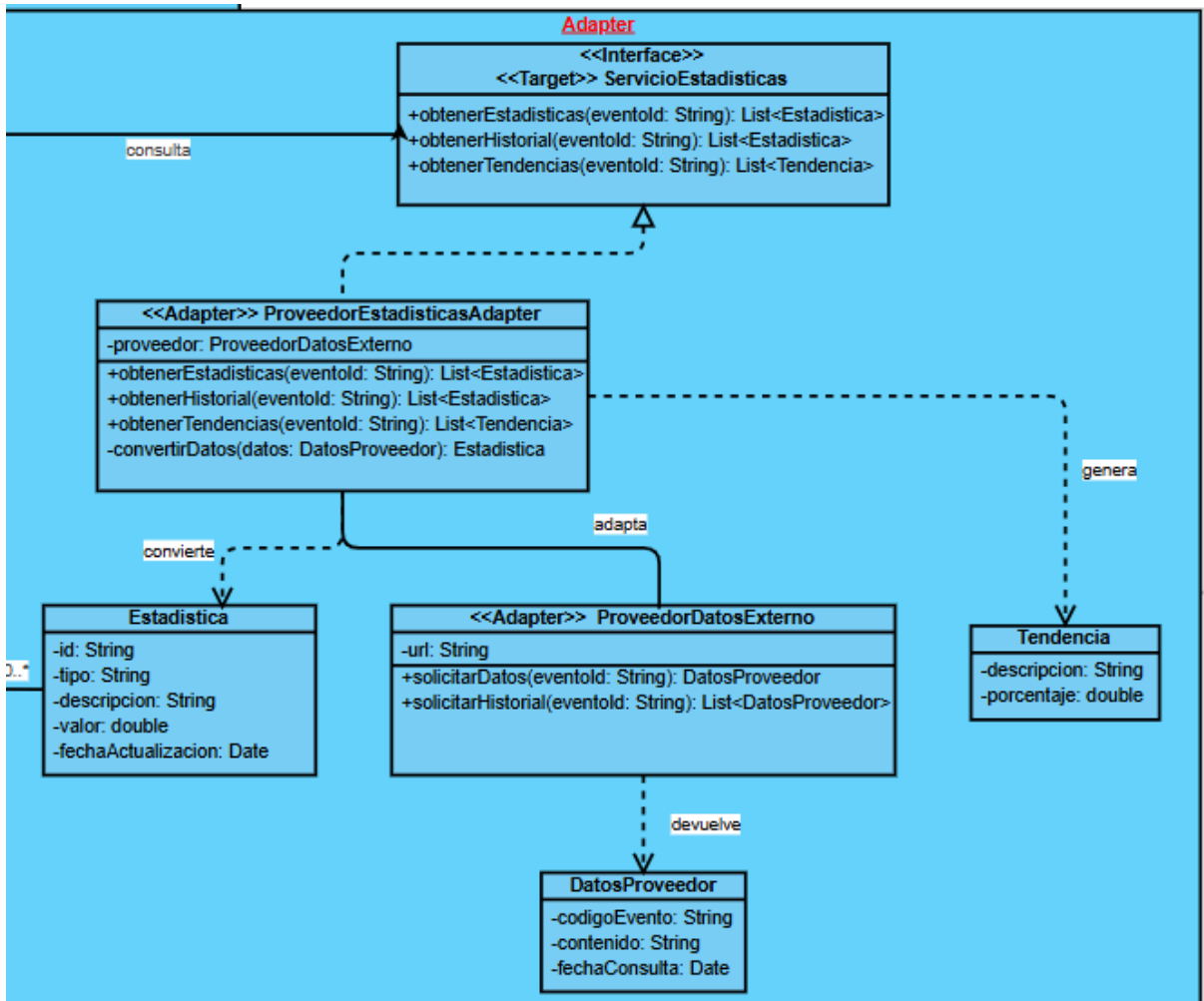


Figura 4: Adapter

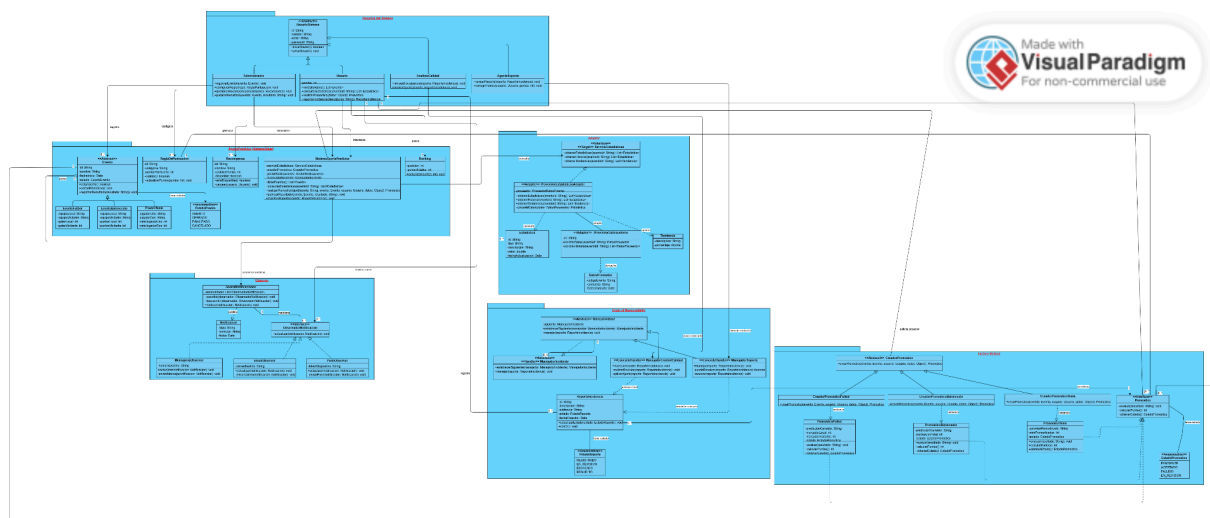


Figura 5: Sistema Completo